

OpenClaw 速成手册

从入门到精通

作者：@开发者阿橙

微信公众号：@开发者阿橙

Twitter / X：@clawcampnet

OpenClaw 创业实战营

让AI成为你的赚钱机器



OpenClaw

知识星球

OpenClaw 创业实战营

开发者阿橙

星主



微信扫码加入星球

目录

1. [初识 OpenClaw](#)
 2. [核心特性与架构](#)
 3. [系统要求与准备工作](#)
 4. [安装指南](#)
 5. [快速配置](#)
 6. [连接你的聊天平台](#)
 7. [深入理解记忆系统](#)
 8. [技能系统全攻略](#)
 9. [安全配置指南](#)
 10. [实战应用场景](#)
 11. [故障排除与FAQ](#)
 12. [进阶主题](#)
 13. [社区资源汇总](#)
-

1. 初识 OpenClaw

1.1 什么是 OpenClaw?

OpenClaw 是一个开源的个人 **AI Agent**（智能体）框架，由奥地利开发者 Peter Steinberger 于 2025 年底发布。它在 2026 年 1 月的技术圈爆红，标志着 AI 从"对话工具"向"自主执行者"的范式转变。

核心理念： OpenClaw 不是一个 App，而是一个"住在电脑里的 AI 管家"。它运行在你自己的设备上，通过自然语言指令帮你完成各种任务。

1.2 创始人背景

Peter Steinberger 是奥地利知名连续创业者：

- **职业经历：** 曾创立 PSPDFKit 公司，开发出全球领先的 iOS PDF 处理框架，客户包括苹果、Adobe 等科技巨头
- **退休与回归：** 2021 年以约 1.19 亿美元出售 PSPDFKit 股份后宣布退休，四年后因"创作欲望"重返技术领域
- **开发理念：** OpenClaw 最初只是他 2025 年底的"周末项目"，初衷是为自己打造一款能自动化处理日常事务的工具，没想到在 GitHub 发布后迅速爆红

Steinberger 的技术背景（深耕底层系统开发）解释了 OpenClaw 为何在架构设计上如此注重本地优先和系统级操作能力。

1.3 命名变迁史

OpenClaw 的命名经历了三天两次改名的传奇：

第一阶段：Clawbot / Clawdbot（2025年11月-2026年1月26日）

- **Claw**：龙虾的爪子，致敬 Anthropic Claude 加载界面中的卡通龙虾形象
- **Clawd**：与 Claude 谐音，暗示项目基于 Claude 模型构建

爆红数据：发布后 24 小时内 GitHub 斩获 9,000 Star，一周内突破 60,000 Star

第二阶段：Moltbot（2026年1月27日-1月29日）

- **改名原因**：Anthropic 法务团队于 1 月 27 日发出"友好建议函"，指出 Clawd/Clawdbot 与 Claude 商标过于相似，存在侵权风险
- **决策过程**：Steinberger 在凌晨 5 点的 Discord 会议中决定改名

第三阶段：OpenClaw（2026年1月29日至今）

- **最终定名**：Open（开源）+ Claw（保留原有意象）
- **寓意**：开放式智能助手，延续社区共建理念

1.4 为什么选择 OpenClaw?

与传统 AI 聊天机器人相比，OpenClaw 的独特优势：

特性	OpenClaw	ChatGPT/Claude网页版	传统聊天机器人
数据隐私	✅ 本地存储，完全可控	❌ 云端存储	⚠️ 取决于服务商
长期记忆	✅ 持久化记忆系统	⚠️ 会话级别	❌ 通常无记忆
自主执行	✅ 可执行实际任务	❌ 仅生成文本	❌ 仅响应指令
多平台集成	✅ 支持十几种平台	❌ 仅网页/App	⚠️ 平台受限
可扩展性	✅ 技能系统，无限扩展	❌ 功能固定	⚠️ 取决于服务商
成本	⚠️ 需自付API费用	✅ 免费版可用	✅ 通常免费

2. 核心特性与架构

2.1 六大核心特性

1. 本地优先 (Local-First)

所有数据存储在你的设备上，包括：

- 会话记录
- 记忆文件
- 配置信息
- 技能脚本

优势：

- 完全的数据所有权
- 隐私安全可控
- 无需担心服务商关闭服务
- 可随时备份和迁移

2. 多渠道通信 (Multi-Channel)

官方支持的平台：

-  WhatsApp (QR码登录，体验最佳)
-  Telegram (推荐新手)
-  Discord
-  Slack
-  iMessage (macOS)
-  Signal
-  Mattermost (通过插件)

社区扩展支持：

-  飞书 (第三方插件)
-  钉钉 (第三方插件)
-  微信 (开发中)

3. 持久记忆系统 (Persistent Memory)

OpenClaw 的突破性功能——真正的长期记忆：

```
memory/  
├── 2026-01-15.md          # 每日记忆日记  
└── 2026-01-16.md
```

```
|— ...
|— MEMORY.md          # 长期记忆精华 (AI自动整理)

workspace/
|— SOUL.md            # AI人格定义
|— USER.md            # 用户画像
```

工作原理：

1. 每次对话生成当天的记忆日记
2. AI 定期将重要信息提炼到 MEMORY.md
3. 跨会话自动读取相关记忆
4. 语义搜索相关历史记录

4. 技能生态系统 (Skills System)

技能是 OpenClaw 的"手脚"，让它能执行具体任务：

内置技能示例：

- web-search：Brave 搜索
- browser：网页自动化
- terminal：命令执行
- code：代码运行
- memory：记忆管理

社区技能 (1700+)：

- 邮件处理
- 日程管理
- 社交媒体发布
- 数据分析
- 文件处理
- API 集成

5. 透明可控 (Transparent & Controllable)

完全的开源透明：

- 所有代码开源在 GitHub
- 可以审查每一行代码
- 可以修改任何行为
- 社区驱动开发

配置即代码：

```
{
  "agents": {
    "main": {
      "model": "claude-sonnet-4-5-20250929",
      "memory": {
        "enabled": true,
        "maxTokens": 8000
      },
      "tools": ["web", "browser", "terminal"]
    }
  }
}
```

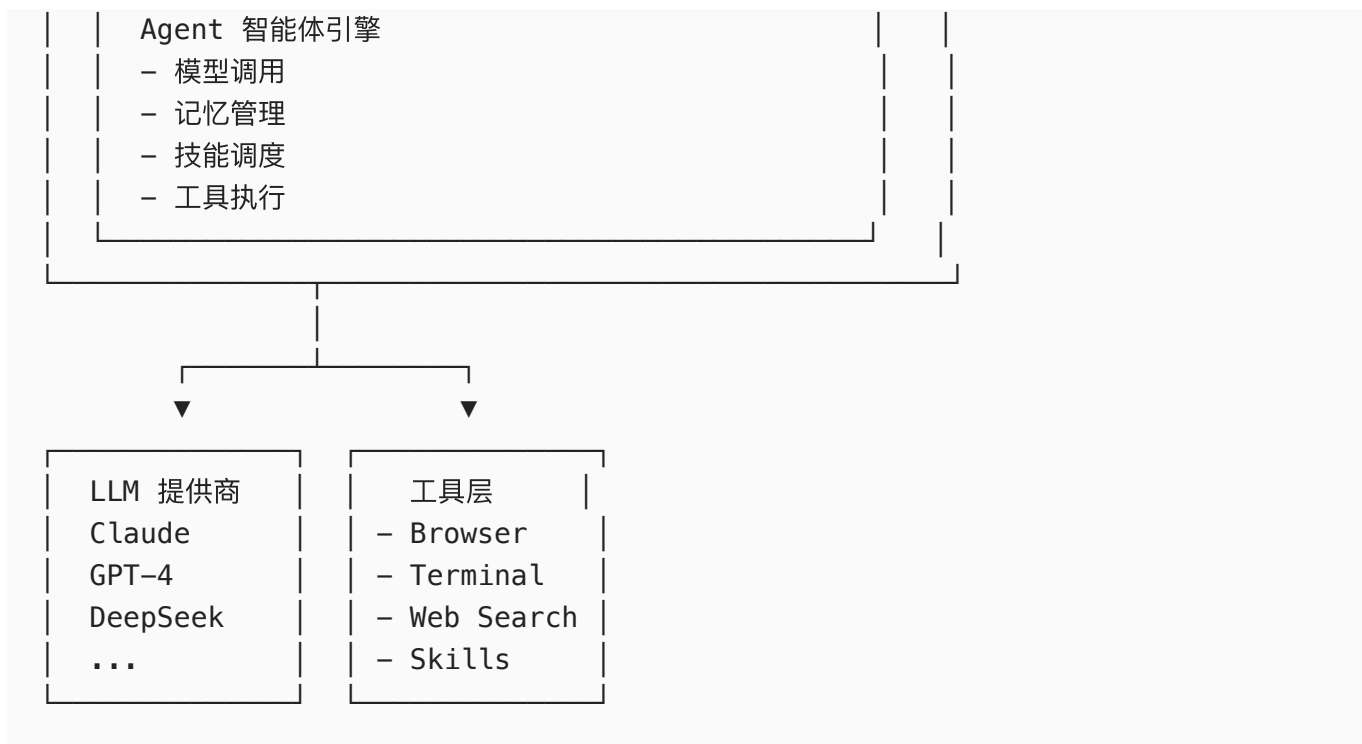
6. AI 模型灵活选择

支持的模型提供商：

- ☒ Anthropic Claude（推荐）
- ☒ OpenAI GPT系列
- ☒ Google Gemini
- ☒ DeepSeek（高性价比）
- ☒ 月之暗面 Kimi
- ☒ 百度千帆
- ☒ 阿里百炼
- ☒ OpenRouter（聚合平台）
- ☒ 本地模型（Ollama、LM Studio）

2.2 系统架构





3. 系统要求与准备工作

3.1 最低系统要求

macOS

- 系统版本： macOS 12 (Monterey) 或更高
- 处理器： Intel 第6代或 Apple M1及以上
- 内存： 8GB（推荐16GB）
- 存储： 至少10GB可用空间
- 网络： 稳定的互联网连接

Linux（推荐 Ubuntu 22.04+）

- 系统版本： Ubuntu 22.04 LTS / Debian 11+ / CentOS 8+
- 处理器： x86_64 或 ARM64
- 内存： 4GB（推荐8GB）
- 存储： 至少10GB可用空间
- 网络： 稳定的互联网连接

Windows

- 推荐方式： WSL2（Windows Subsystem for Linux 2）

- 系统版本： Windows 10 2004 或更高 / Windows 11
- 处理器： 支持虚拟化
- 内存： 8GB（推荐16GB）
- 存储： 至少20GB可用空间

⚠ **重要提示：** 原生 Windows 支持有限，强烈建议使用 WSL2！

3.2 软件依赖

必需软件

```
# Node.js (必需, 版本 >= 22)
node --version # 应显示 v22.x.x 或更高

# npm 或 pnpm (包管理器)
npm --version # 应显示 10.x.x 或更高
# 或
pnpm --version # 应显示 8.x.x 或更高
```

可选软件

```
# Git (从源码安装需要)
git --version

# Docker (容器化部署, 推荐)
docker --version

# Python (某些技能可能需要)
python --version
```

3.3 准备工作清单

步骤1：安装 Node.js

macOS/Linux：

```
# 使用 nvm (推荐)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh |
bash
source ~/.bashrc # 或 source ~/.zshrc
nvm install 22
nvm use 22
```

```
# 或使用包管理器
# macOS (Homebrew)
brew install node@22

# Ubuntu/Debian
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Windows (WSL2):

```
# 在 WSL2 中执行
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

步骤2：验证安装

```
node --version
npm --version
```

步骤3：获取 API Key

你需要至少一个 LLM 提供商的 API Key:

Anthropic Claude（推荐）:

1. 访问 <https://console.anthropic.com>
2. 注册/登录账号
3. 进入 API Keys 页面
4. 创建新的 API Key
5. ⚠️ 重要：妥善保存 API Key，只显示一次！

OpenAI GPT:

1. 访问 <https://platform.openai.com>
2. 注册/登录
3. API Keys → Create new secret key

DeepSeek（高性价比）:

1. 访问 <https://platform.deepseek.com>
2. 注册/登录
3. API Keys → 创建密钥

月之暗面 Kimi:

1. 访问 <https://platform.moonshot.cn>
2. 注册/登录
3. API Key 管理

步骤4：申请聊天平台凭证

Telegram Bot（推荐新手）:

1. 在 Telegram 中搜索 @BotFather
2. 发送 /newbot
3. 按提示设置机器人名称
4. 保存返回的 API Token

Discord Bot:

1. 访问 <https://discord.com/developers/applications>
2. 创建应用 → Bot → 创建 Bot
3. 保存 Bot Token

飞书（国内推荐）:

1. 访问飞书开放平台 <https://open.feishu.cn>
2. 创建企业自建应用
3. 获取 App ID 和 App Secret
4. 配置事件订阅

作者：OpenClaw创业实战营

知识星球： <https://t.zsxq.com/OOGnZ>

微信公众号： @开发者阿橙

版本时间： 2026年2月版

4. 安装指南

4.1 方法一：官方安装脚本（推荐）

这是最简单、最快速的安装方式，适合大多数用户。

macOS / Linux:

```
# 一键安装
curl -fsSL https://openclaw.ai/install.sh | bash
```

Windows (PowerShell):

```
# 管理员权限运行 PowerShell
iwr -useb https://openclaw.ai/install.ps1 | iex
```

安装过程说明:

1. 自动检测系统环境
2. 下载最新版本的 OpenClaw CLI
3. 安装到系统路径（通常在 `/usr/local/bin` 或 `~/.local/bin`）
4. 配置环境变量
5. 显示安装成功信息

验证安装:

```
openclaw --version
# 应显示类似: OpenClaw CLI v2026.2.8
```

4.2 方法二: npm 全局安装

如果你熟悉 Node.js 生态系统，可以直接通过 npm 安装:

```
# 使用 npm
npm install -g openclaw@latest

# 或使用 pnpm (更快, 更节省空间)
pnpm add -g openclaw@latest
```

验证安装:

```
openclaw --version
```

4.3 方法三: 从源码安装 (开发者推荐)

适合想要深度定制或参与开发的用户:

```
# 1. 克隆仓库
git clone https://github.com/openclaw/openclaw.git
```

```
cd openclaw

# 2. 安装依赖
pnpm install

# 3. 构建 UI 资源
pnpm ui:build

# 4. 构建项目
pnpm build

# 5. 运行新手向导
pnpm openclaw onboard --install-daemon
```

4.4 方法四：Docker 部署（服务器推荐）

Docker 部署适合云服务器和需要隔离环境的场景：

创建 **docker-compose.yml**：

```
version: '3.8'

services:
  openclaw:
    image: openclaw/openclaw:latest
    container_name: openclaw
    restart: unless-stopped

    # 端口映射
    ports:
      - "18789:18789" # Gateway 网关端口
      - "3000:3000"   # Dashboard 端口（可选）

    # 环境变量
    environment:
      - NODE_ENV=production
      - TZ=Asia/Shanghai

    # 数据卷挂载
    volumes:
      - ./openclaw-data:/home/node/.openclaw
      - /var/run/docker.sock:/var/run/docker.sock # Docker 沙箱

    # Docker 沙箱支持
    privileged: true
```

```
# 网络模式
network_mode: "bridge"
```

启动容器：

```
# 启动服务
docker-compose up -d

# 查看日志
docker-compose logs -f openclaw

# 进入容器
docker-compose exec openclaw bash
```

配置 OpenClaw（容器内）：

```
# 进入容器后运行
openclaw onboard
```

4.5 方法五：云平台一键部署（适合非技术用户）

阿里云轻量应用服务器：

1. 登录阿里云控制台
2. 选择"轻量应用服务器"
3. 镜像选择"OpenClaw"
4. 购买实例（最低配置：2核4GB）
5. 等待部署完成（约5-10分钟）
6. 获取登录密码和访问地址

腾讯云轻量应用服务器：

1. 登录腾讯云控制台
2. 选择"轻量应用服务器"
3. 应用镜像选择"OpenClaw"
4. 创建实例
5. 通过 SSH 或控制台访问

4.6 安装验证

无论使用哪种安装方式，最后都需要验证：

```
# 1. 检查版本
openclaw --version

# 2. 查看系统状态
openclaw status

# 3. 健康检查
openclaw health

# 4. 安全审计
openclaw security audit --deep
```

预期输出示例：

```
✓ OpenClaw CLI: v2026.2.8
✓ Node.js: v22.1.0
✓ Gateway: 运行中 (PID: 12345)
✓ 内存使用: 245MB
✓ 端口 18789: 开放
```

5. 快速配置

5.1 使用新手向导（推荐）

OpenClaw 提供了交互式配置向导，5分钟内完成所有配置：

```
openclaw onboard --install-daemon
```

向导会引导你完成：

步骤1：选择配置模式

```
? 选择配置模式：
  本地模式（推荐新手）
  远程服务器模式
  高级自定义模式
```

步骤2：配置 AI 模型

? 选择 AI 提供商:
Anthropic Claude (推荐)
OpenAI GPT-4
Google Gemini
DeepSeek (高性价比)
月之暗面 Kimi
OpenRouter (聚合)
本地模型 (Ollama)

? 输入 API Key:
sk-ant-xxxxx...

步骤3: 配置 Gateway

? Gateway 端口:
18789 (默认)

? 启用 Token 认证?
Yes (推荐)

? 生成 Token (自动)
[系统生成随机Token并保存]

步骤4: 配置渠道

? 选择聊天平台:
Telegram (推荐新手)
Discord
WhatsApp
飞书 (需第三方插件)

步骤5: 安装守护进程

? 安装系统服务?
Yes (推荐后台运行)

? 运行时:
Node (推荐)
Bun (不推荐 WhatsApp/Telegram)

步骤6: 配置工作区

? 配置默认技能:

web-search
memory-manager
code-executor

? 创建工作区?

~/.openclaw/workspace (默认)

5.2 手动配置 (高级用户)

如果你想完全控制配置细节, 可以手动编辑配置文件。

配置文件位置

```
~/.openclaw/  
├─ openclaw.json          # 主配置文件  
├─ agents/  
│   └─ main/  
│       ├── agent.json    # Agent 配置  
│       └─ auth-profiles.json # 认证配置  
├─ credentials/  
│   └─ oauth.json        # OAuth 凭证 (如果使用)  
└─ workspace/            # 工作区目录  
    ├── SOUL.md  
    └─ USER.md
```

主配置文件示例 (openclaw.json):

```
{  
  "gateway": {  
    "port": 18789,  
    "host": "127.0.0.1",  
    "auth": {  
      "token": "your-secure-token-here"  
    }  
  },  
  "agents": {  
    "main": {  
      "workspace": "~/.openclaw/workspace",  
      "model": {  
        "provider": "anthropic",  
        "model": "claude-sonnet-4-5-20250929"  
      },  
      "memory": {
```

```

        "enabled": true,
        "maxTokens": 8000,
        "storagePath": "~/.openclaw/memory"
    },
    "tools": [
        "web",
        "browser",
        "terminal",
        "memory"
    ],
    "sandbox": {
        "mode": "non-main"
    }
}
},
"channels": {
    "telegram": {
        "enabled": true,
        "botToken": "your-telegram-bot-token"
    },
    "discord": {
        "enabled": false
    }
}
}
}

```

API 认证配置：

方式1：API Key（推荐 Anthropic）：

```

# 使用命令行配置
openclaw configure --section models.anthropic.apiKey
# 然后粘贴你的 API Key

# 或直接编辑文件
~/.openclaw/agents/main/agent/auth-profiles.json

```

```

{
  "profiles": {
    "default": {
      "providers": {
        "anthropic": {
          "apiKey": "sk-ant-api03-xxxxx..."
        }
      }
    }
  }
}

```

```
}  
}  
}
```

方式2: OpenAI Code OAuth (实验性):

```
# 在本地机器上完成 OAuth  
openclaw configure --oauth openai  
  
# 复制 oauth.json 到服务器  
scp ~/.openclaw/credentials/oauth.json user@server:~/.openclaw/credentials/
```

5.3 配置 Web 搜索 (强烈推荐)

Brave Search 为 OpenClaw 提供联网能力:

```
# 1. 获取 Brave Search API Key  
# 访问 https://brave.com/search/api/  
  
# 2. 配置 OpenClaw  
openclaw configure --section web  
# 输入 API Key  
  
# 3. 验证  
openclaw test web-search
```

5.4 配置 Docker 沙箱 (安全推荐)

如果使用 Docker, 配置沙箱增强安全性:

```
{  
  "agents": {  
    "main": {  
      "sandbox": {  
        "mode": "docker",  
        "docker": {  
          "image": "openclaw/sandbox:latest",  
          "memoryLimit": "512m",  
          "cpuLimit": "1.0"  
        }  
      }  
    }  
  }  
}
```

```
}  
}
```

5.5 启动 Gateway

方式1：系统服务（推荐）

如果向导中安装了守护进程：

```
# macOS  
launchctl start openclaw-gateway  
  
# Linux  
systemctl start openclaw-gateway  
systemctl enable openclaw-gateway # 开机自启  
  
# 查看状态  
launchctl status openclaw-gateway # macOS  
systemctl status openclaw-gateway # Linux
```

方式2：手动启动

```
# 前台运行（调试用）  
openclaw gateway --port 18789 --verbose  
  
# 后台运行  
nohup openclaw gateway --port 18789 > openclaw.log 2>&1 &
```

访问 Dashboard

```
# 本地访问  
http://127.0.0.1:18789/  
  
# 如果配置了 Token，输入：  
your-secure-token-here
```

5.6 配置验证

```
# 1. 检查状态  
openclaw status  
  
# 2. 健康检查  
openclaw health
```

```
# 3. 深度诊断
openclaw status --deep

# 4. 安全审计
openclaw security audit --deep
```

成功标志：

- ✓ Gateway 运行中
- ✓ 已配置认证
- ✓ 端口正常监听
- ✓ Agent 可用
- ✓ 无安全警告

6. 连接你的聊天平台

6.1 Telegram（推荐新手）

Telegram 是最适合新手的平台，配置简单，功能完整。

步骤1：创建 Bot

1. 在 Telegram 中搜索 @BotFather
2. 发送 /newbot 命令
3. 按提示设置：
 - Bot 名称：MyOpenClawBot
 - 用户名：MyOpenClawBot_bot（必须以 _bot 结尾）
4. 保存返回的 API Token：

```
123456789:ABCdefGhIJKlmnoPQRStuvWXYZ
```

步骤2：配置 OpenClaw

方式1：使用向导（推荐）：

```
openclaw onboard
# 选择 Telegram
# 粘贴 Bot Token
```

方式2：手动配置：

编辑 ~/.openclaw/openclaw.json :

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "botToken": "123456789:ABCdefGhIJKlmnoPQRStuvWXYZ"
    }
  }
}
```

步骤3: 重启 Gateway

```
# 如果使用系统服务
systemctl restart openclaw-gateway

# 或手动重启
openclaw gateway --restart
```

步骤4: 首次配对

1. 在 Telegram 中搜索你的 Bot
2. 发送任意消息, 如 /start
3. Bot 会返回一个配对码, 如:

你的配对码是: ABC123
请在 OpenClaw 中批准此配对

4. 在终端中执行:

```
openclaw pairing list telegram
openclaw pairing approve telegram ABC123
```

步骤5: 开始聊天

现在你可以通过 Telegram 与 OpenClaw 对话了!

测试命令:

```
/start - 开始对话
/help - 查看帮助
/remember <内容> - 记住重要信息
/forget - 查看记忆
```

6.2 Discord

Discord 适合社区和多人群聊场景。

步骤1：创建 Discord 应用

1. 访问 <https://discord.com/developers/applications>
2. 点击 "New Application"
3. 填写应用名称： OpenClaw
4. 在 "Bot" 页面：
 - 点击 "Add Bot"
 - 复制 Bot Token

步骤2：配置 Bot 权限

在 "OAuth2" → "URL Generator" 页面：

Scopes：

- ☒ bot

Bot Permissions：

- ☒ Read Messages/View Channels
- ☒ Send Messages
- ☒ Embed Links
- ☒ Attach Files
- ☒ Add Reactions
- ☒ Use Slash Commands
- ☒ Manage Messages （可选，用于删除消息）

生成邀请链接并邀请 Bot 到你的服务器。

步骤3：配置 OpenClaw

```
openclaw configure --section channels.discord
```

或编辑配置文件：

```
{
  "channels": {
    "discord": {
      "enabled": true,
```

```
    "botToken": "your-discord-bot-token",
    "clientId": "your-client-id"
  }
}
```

步骤4：重启并测试

```
systemctl restart openclaw-gateway
```

在 Discord 中 @你的机器人 发送消息

6.3 WhatsApp（体验最佳）

WhatsApp 提供最接近真实聊天的体验，支持 QR 码登录。

步骤1：配置 OpenClaw

```
openclaw configure --section channels.whatsapp
```

```
{
  "channels": {
    "whatsapp": {
      "enabled": true,
      "businessApi": false # 使用个人 WhatsApp
    }
  }
}
```

步骤2：QR 码登录

启动 Gateway 时会显示 QR 码

```
openclaw gateway --verbose
```

或查看日志获取 QR 码

```
journalctl -u openclaw-gateway -f
```

步骤3：扫码配对

1. 在 WhatsApp 手机应用中
2. 设置 → 链接设备
3. 扫描终端显示的 QR 码

⚠ 重要提示:

- WhatsApp 需要 **Node.js** 运行时 (不支持 Bun)
- 需要保持 Gateway 持续运行
- QR 码定期失效, 需要重新登录

6.4 飞书 (国内推荐)

飞书插件由社区维护, 需要额外安装。

步骤1: 安装飞书插件

```
# 克隆飞书插件仓库
git clone https://github.com/AlexAnys/openclaw-feishu.git
cd openclaw-feishu

# 安装依赖
npm install

# 构建
npm run build
```

步骤2: 创建飞书应用

1. 访问 <https://open.feishu.cn>
2. 创建企业自建应用
3. 获取 App ID 和 App Secret
4. 配置事件订阅和权限

步骤3: 配置插件

```
# 复制配置文件
cp config.example.json config.json

# 编辑配置
nano config.json
```

```
{
  "feishu": {
    "appId": "cli_xxxxxxxx",
    "appSecret": "xxxxxxxxxxxxxxxx",
    "encryptKey": "xxxxxxxxxxxxxxxx",
    "verificationToken": "xxxxxxxxxxxxxxxx"
```

```
    },  
    "openclaw": {  
      "gatewayUrl": "http://127.0.0.1:18789",  
      "token": "your-gateway-token"  
    }  
  }  
}
```

步骤4：启动插件

```
# 启动飞书插件服务  
npm start  
  
# 或使用 PM2  
pm2 start npm --name "openclaw-feishu" -- start
```

步骤5：测试

在飞书中向应用发送消息，应该能收到 OpenClaw 的回复。

详细教程：

- <https://github.com/AlexAnys/openclaw-feishu>
- <https://www.cnblogs.com/catchadmin/p/19556552>

6.5 多平台并行

OpenClaw 支持同时连接多个平台：

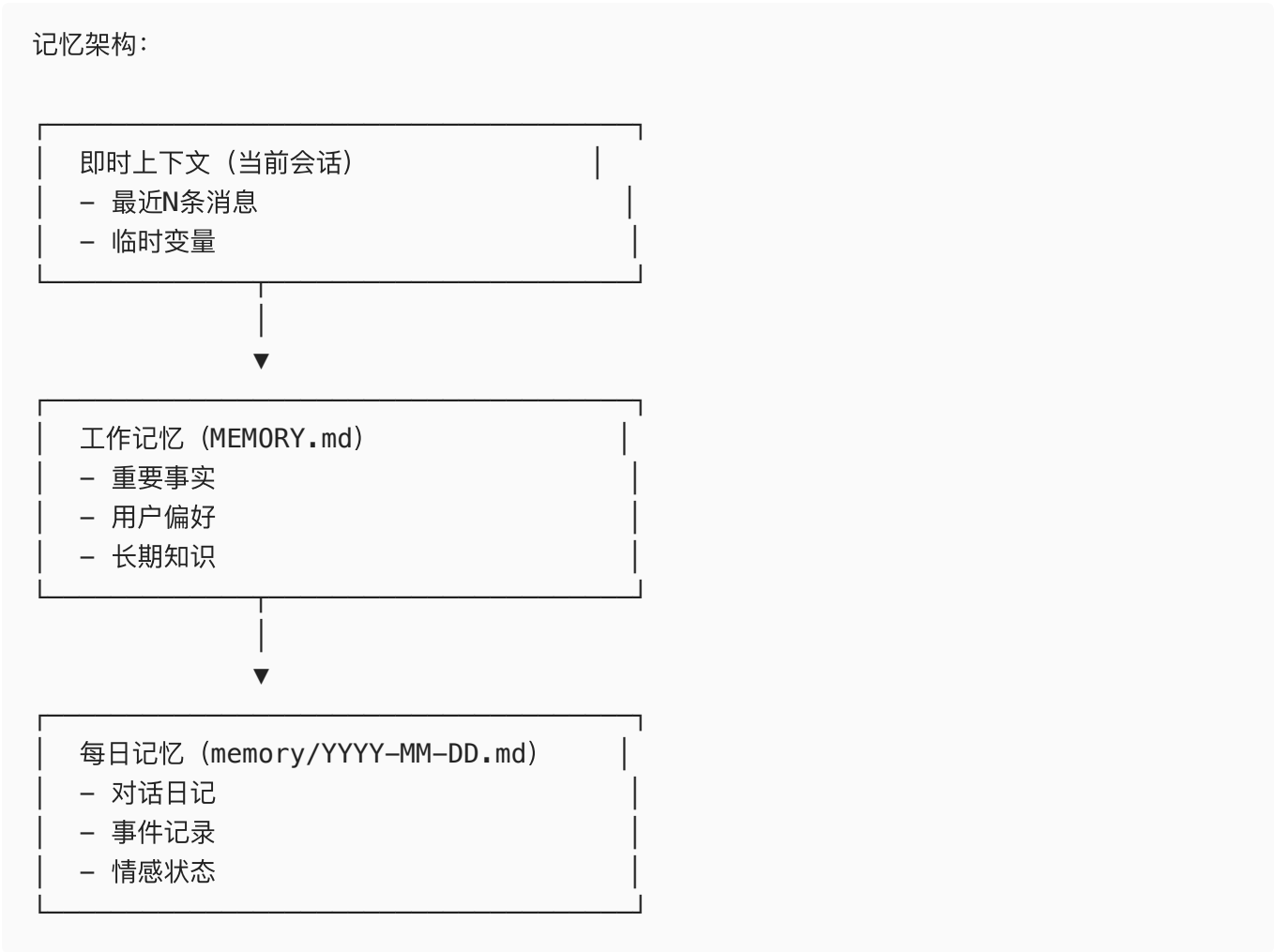
```
{  
  "channels": {  
    "telegram": {  
      "enabled": true,  
      "botToken": "..."  
    },  
    "discord": {  
      "enabled": true,  
      "botToken": "..."  
    },  
    "whatsapp": {  
      "enabled": true  
    }  
  }  
}
```

提示： 所有平台共享同一个 Agent、记忆和工作区，提供一致的体验。

7. 深入理解记忆系统

7.1 记忆系统架构

OpenClaw 的记忆系统是其核心竞争力，让 AI 真正"记住"你。



7.2 记忆文件详解

MEMORY.md (长期记忆)

这是 AI 的"长期记忆"，包含最精华的信息：

```
# 用户：张三

## 基本信息
- 职业：软件工程师
- 位置：北京
```

- 时区: GMT+8

偏好

- 编程语言: Python, JavaScript
- 工作时间: 9:00-18:00
- 沟通风格: 简洁直接

重要事件

- 2026-01-15: 完成 OpenClaw 部署
- 2026-01-20: 通过 Telegram 渠道使用

待办事项

- [] 学习技能开发
- [] 配置更多渠道

每日记忆 (memory/YYYY-MM-DD.md)

每天一个文件, 记录当天的对话:

2026-01-15

对话摘要

- 上午: 配置 WhatsApp 渠道
- 下午: 测试 Web 搜索功能
- 晚上: 记忆系统测试

重要信息

- 用户喜欢使用 Claude Sonnet 模型
- API 预算: 每月\$50
- 关注安全配置

情感状态

- 对新工具感到兴奋
- 略感配置复杂

SOUL.md (AI 人格定义)

定义 AI 的行为准则和性格:

OpenClaw 助手

使命

帮助用户提高效率, 自动化日常任务。

行为准则

- 始终诚实透明
- 尊重用户隐私
- 主动提供建议
- 承认不确定性

沟通风格

- 专业但不拘谨
- 适当使用幽默
- 中文交流为主

禁止事项

- 不生成有害内容
- 不泄露系统信息
- 不执行危险操作

USER.md（用户画像）

AI 对用户的理解和建模：

用户画像

技术水平

- 高级用户
- 熟悉命令行
- 有编程背景

交互偏好

- 喜欢简洁回复
- 偏好文本格式
- 不需要过多解释

常见任务

- 代码调试
- 文档编写
- 数据分析

7.3 记忆更新机制

OpenClaw 采用"被动+主动"混合的记忆更新策略：

被动记忆（用户主动输入）

用户：记住我喜欢在上午10点开会

AI：好的，我记住了：你喜欢在上午10点开会

```
# 自动更新到 MEMORY.md
```

主动记忆 (AI 自主判断)

OpenClaw 会根据对话内容自主判断哪些信息值得记住：

```
// AI 内部决策过程 (简化)
if (信息重要性 > 阈值) {
    写入MEMORY.md(信息)
    if (需要详细信息) {
        写入每日日记()
    }
}
```

定期记忆整理

AI 会定期整理记忆文件：

1. **去重**：移除重复信息
2. **归类**：将相关信息分组
3. **优先级**：标记重要程度
4. **归档**：将旧信息移至历史档案

7.4 记忆搜索

OpenClaw 使用语义搜索查找相关记忆：

语义搜索

用户：我的项目进度如何？

AI 搜索过程：

1. 分析查询意图 → "项目", "进度"
2. 语义搜索记忆库
3. 找到相关记忆：
 - "2026-01-10: 开始OpenClaw项目"
 - "2026-01-12: 完成配置"
 - "2026-01-15: 开始测试"

AI 回复：根据记忆，你的OpenClaw项目目前进展顺利...

关键词搜索

用户：搜索关于"API"的所有记忆

AI：找到5条相关记忆...

7.5 记忆管理最佳实践

1. 定期审查记忆

```
# 查看记忆文件
cat ~/.openclaw/workspace/MEMORY.md

# 查看最近的每日记忆
ls -lt ~/.openclaw/memory/
```

2. 手动编辑记忆

你可以直接编辑记忆文件：

```
# 编辑长期记忆
nano ~/.openclaw/workspace/MEMORY.md

# 编辑AI人格
nano ~/.openclaw/workspace/SOUL.md
```

3. 记忆备份

```
# 备份记忆目录
cp -r ~/.openclaw ~/.openclaw-backup-$(date +%Y%m%d)

# 或使用 Git
cd ~/.openclaw
git add workspace/ memory/
git commit -m "更新记忆"
```

4. 记忆清理

```
# 清理旧的记忆文件（保留最近30天）
find ~/.openclaw/memory/ -name "*.md" -mtime +30 -delete
```

7.6 高级记忆配置

调整记忆容量

```

{
  "agents": {
    "main": {
      "memory": {
        "enabled": true,
        "maxTokens": 12000,          // 增加记忆容量
        "summaryThreshold": 5000,    // 超过5000 token开始总结
        "compressionRatio": 0.3      // 压缩到30%
      }
    }
  }
}

```

自定义记忆策略

```

{
  "agents": {
    "main": {
      "memory": {
        "strategy": "semantic",
        "updateInterval": "auto",
        "retentionDays": 90,
        "importantKeywords": [
          "重要", "紧急", "记住",
          "偏好", "配置", "密钥"
        ]
      }
    }
  }
}

```

8. 技能系统全攻略

8.1 技能系统概述

技能（Skills）是 OpenClaw 的"手脚"，让它能够执行具体任务。

技能层次：

内置技能（核心）

- └─ web-search：联网搜索
- └─ browser：网页自动化

- └─ terminal: 命令执行
- └─ code: 代码运行
- └─ memory: 记忆管理

官方技能

- └─ calendar: 日程管理
- └─ email: 邮件处理
- └─ notes: 笔记管理
- └─ file: 文件操作

社区技能 (1700+)

- └─ social-media: 社交媒体
- └─ analytics: 数据分析
- └─ automation: 自动化
- └─ custom: 用户自定义

8.2 内置技能详解

web-search (联网搜索)

功能： 让 AI 能够搜索实时信息

配置：

```
# 获取 Brave Search API Key
openclaw configure --section web

# 输入 API Key
```

使用示例：

```
用户：搜索最新的 OpenClaw 更新
AI：[正在搜索...]
    根据搜索结果，OpenClaw 最新版本是 v2026.2.8...
```

高级配置：

```
{
  "tools": {
    "web": {
      "search": {
        "provider": "brave",
        "apiKey": "your-api-key",
        "maxResults": 10,
```

```
        "timeout": 10000
      }
    }
  }
}
```

browser（网页自动化）

功能： 浏览、交互、操作网页

依赖： Puppeteer/Playwright

使用示例：

用户：打开 GitHub OpenClaw 仓库并截图
AI：[正在打开页面...]
[截图已生成]

配置：

```
{
  "tools": {
    "browser": {
      "enabled": true,
      "headless": true,
      "viewport": {
        "width": 1920,
        "height": 1080
      }
    }
  }
}
```

terminal（命令执行）

功能： 执行系统命令

⚠️ **安全警告：** 此技能具有系统级访问权限，请谨慎使用！

配置：

```
{
  "tools": {
    "terminal": {
```

```
    "enabled": true,
    "allowedCommands": [
      "ls", "cd", "cat", "grep",
      "git", "npm", "node"
    ],
    "blockedCommands": [
      "rm -rf /",
      "sudo",
      "chmod 777"
    ]
  }
}
```

使用示例：

用户：列出当前目录的文件

AI：[执行命令：ls -la]

total 24

drwxr-xr-x 5 user staff 160 Jan 15 10:00 .

drwxr-xr-x 3 user staff 96 Jan 15 09:55 ..

-rw-r--r-- 1 user staff 1234 Jan 15 10:00 file.txt

code（代码执行）

功能： 执行代码片段

支持语言：

- JavaScript/Node.js
- Python（需配置）
- 其他语言（通过扩展）

使用示例：

用户：执行这段代码：

```
console.log("Hello, OpenClaw!");
```

AI：[执行中...]

输出：Hello, OpenClaw!

8.3 安装社区技能

方法1：通过 **ClawHub**（官方技能市场）

```
# 搜索技能
openclaw skills search "email"

# 安装技能
openclaw skills install email-manager

# 列出已安装技能
openclaw skills list

# 更新技能
openclaw skills update email-manager

# 卸载技能
openclaw skills uninstall email-manager
```

方法2：从 GitHub 安装

```
# 克隆技能仓库
git clone https://github.com/user/openclaw-skill-email.git
cd openclaw-skill-email

# 安装依赖
npm install

# 复制到技能目录
cp -r . ~/.openclaw/skills/email-manager

# 重启 Gateway
systemctl restart openclaw-gateway
```

方法3：手动创建技能

技能目录结构：

```
~/.openclaw/skills/my-skill/
├─ skill.json          # 技能元数据
├─ README.md          # 技能文档
├─ index.js           # 技能逻辑
└─ assets/            # 资源文件
```

skill.json 示例：

```

{
  "name": "my-skill",
  "version": "1.0.0",
  "description": "我的自定义技能",
  "author": "Your Name",
  "main": "index.js",
  "permissions": [
    "web",
    "terminal"
  ],
  "parameters": {
    "apiKey": {
      "type": "string",
      "required": true,
      "description": "API密钥"
    }
  }
}

```

index.js 示例：

```

async function execute(context) {
  const { message, config, tools } = context;

  // 解析用户输入
  const userInput = message.text;

  // 执行技能逻辑
  const result = await doSomething(userInput);

  // 返回结果
  return {
    text: `执行结果: ${result}`,
    attachments: []
  };
}

module.exports = { execute };

```

8.4 推荐技能清单

生产力类

1. calendar-sync

- 功能：日历同步和管理
- 安装： `openclaw skills install calendar-sync`
- 配置：Google Calendar / Outlook API

2. **task-manager**

- 功能：任务清单管理
- 支持：Todoist / TickTick / 本地任务

3. **email-assistant**

- 功能：邮件分类和回复建议
- 支持：Gmail / Outlook / IMAP

社交媒体类

4. **twitter-bot**

- 功能：Twitter 自动发布
- 特性：定时发布、数据分析

5. **telegram-channel-manager**

- 功能：Telegram 频道管理
- 特性：自动转发、内容过滤

开发工具类

6. **github-assistant**

- 功能：GitHub 仓库管理
- 特性：Issue分类、PR审核

7. **code-reviewer**

- 功能：代码审查助手
- 特性：自动评论、建议优化

数据分析类

8. **spreadsheet-analyst**

- 功能：Excel/Google Sheets 分析
- 特性：数据可视化、趋势分析

9. **web-scraper**

- 功能：网页数据抓取
- 特性：批量抓取、数据导出

8.5 技能开发教程

创建一个简单的天气技能

步骤1：初始化技能

```
mkdir -p ~/.openclaw/skills/weather-skill
cd ~/.openclaw/skills/weather-skill
npm init -y
```

步骤2：创建 skill.json

```
{
  "name": "weather-skill",
  "version": "1.0.0",
  "description": "查询天气信息",
  "main": "index.js",
  "permissions": ["web"],
  "parameters": {
    "apiKey": {
      "type": "string",
      "required": true,
      "description": "天气API密钥"
    },
    "defaultCity": {
      "type": "string",
      "required": false,
      "description": "默认城市"
    }
  }
}
```

步骤3：创建 index.js

```
const axios = require('axios');

async function execute(context) {
  const { message, config, tools } = context;

  // 解析用户输入
  const city = extractCity(message.text) || config.defaultCity;

  // 调用天气API
  const weather = await getWeather(city, config.apiKey);

  // 格式化输出
  const response = formatWeather(weather);
```

```

    return {
      text: response,
      attachments: []
    };
  }

  function extractCity(text) {
    // 简单的城市名提取
    const match = text.match(/(?:查询|天气)\s*([^\s,。!?\s]+)/);
    return match ? match[1] : null;
  }

  async function getWeather(city, apiKey) {
    const url = `https://api.weatherapi.com/v1/current.json?key=${apiKey}&q=${city}`;
    const response = await axios.get(url);
    return response.data;
  }

  function formatWeather(data) {
    const { location, current } = data;
    return `
    🌐 ${location.name}, ${location.country}
    🌡️ 温度: ${current.temp_c}°C
    ☁️ 天气: ${current.condition.text}
    💨 风速: ${current.wind_kph} km/h
    💧 湿度: ${current.humidity}%
    `.trim();
  }

  module.exports = { execute };

```

步骤4: 配置技能

编辑 ~/.openclaw/openclaw.json :

```

{
  "skills": {
    "weather": {
      "enabled": true,
      "apiKey": "your-weather-api-key",
      "defaultCity": "北京"
    }
  }
}

```


步骤5：测试技能

用户：查询北京天气

AI: 🌐 北京, 中国

🌡️ 温度: 15°C

☁️ 天气: 晴朗

💨 风速: 10 km/h

💧 湿度: 45%

8.6 技能调试

查看技能日志

```
# 实时查看日志
journalctl -u openclaw-gateway -f

# 过滤技能日志
journalctl -u openclaw-gateway | grep "weather-skill"
```

测试技能

```
# 测试单个技能
openclaw skills test weather-skill

# 查看技能详情
openclaw skills info weather-skill
```

常见问题

问题：技能未加载

```
# 检查技能目录权限
ls -la ~/.openclaw/skills/

# 检查 skill.json 语法
cat ~/.openclaw/skills/my-skill/skill.json | jq .
```

问题：技能执行失败

```
# 查看详细错误日志
openclaw logs --skill my-skill --verbose
```

```
# 测试技能依赖
cd ~/.openclaw/skills/my-skill
npm test
```

9. 安全配置指南

9.1 安全风险概述

OpenClaw 是一个强大的 AI Agent，但也带来了一些安全风险：

风险类型	严重性	影响	防护措施
API Key 泄露	🔴 高	经济损失	环境变量、权限限制
未授权访问	🔴 高	数据泄露	Token 认证、IP 白名单
命令注入	🔴 高	系统被控	沙箱隔离、命令白名单
越额消费	🟡 中	意外费用	API 限额、预算告警
数据泄露	🟡 中	隐私暴露	加密存储、访问审计

9.2 API Key 安全

原则1：最小权限

不要给 API Key 过高的权限：

```
{
  "models": {
    "providers": {
      "anthropic": {
        "apiKey": "sk-ant-xxx",
        "permissions": [
          "messages:create",
          "messages:read"
        ],
        "quotas": {
          "maxTokensPerRequest": 100000,
          "maxRequestsPerDay": 1000,
          "maxCostPerMonth": 50
        }
      }
    }
  }
}
```

```
}  
}
```

原则2：定期轮换

```
# 每30天轮换一次 API Key  
openclaw rotate-key --provider anthropic
```

原则3：使用环境变量

```
# ~/.bashrc 或 ~/.zshrc  
export ANTHROPIC_API_KEY="sk-ant-xxx"  
export OPENAI_API_KEY="sk-xxx"  
  
# 在配置文件中引用  
{  
  "models": {  
    "providers": {  
      "anthropic": {  
        "apiKey": "${ANTHROPIC_API_KEY}"  
      }  
    }  
  }  
}
```

9.3 网关认证配置

启用 Token 认证（必须）

```
{  
  "gateway": {  
    "port": 18789,  
    "auth": {  
      "enabled": true,  
      "token": "your-secure-random-token-32-chars-min",  
      "tokenHeader": "X-Gateway-Token"  
    }  
  }  
}
```

生成安全 Token：

```
# 生成32字节随机 Token
openssl rand -hex 32

# 或使用
python3 -c "import secrets; print(secrets.token_hex(32))"
```

IP 白名单

```
{
  "gateway": {
    "auth": {
      "ipWhitelist": [
        "127.0.0.1",
        "192.168.1.0/24",
        "your.server.ip"
      ]
    }
  }
}
```

9.4 Docker 沙箱隔离

配置 Docker 沙箱

```
{
  "agents": {
    "main": {
      "sandbox": {
        "mode": "docker",
        "docker": {
          "image": "openclaw/sandbox:latest",
          "memoryLimit": "512m",
          "cpuLimit": "1.0",
          "networkMode": "bridge",
          "readOnlyRootfs": true,
          "dropCapabilities": ["ALL"],
          "allowedDevices": []
        }
      }
    }
  }
}
```

沙箱模式对比

模式	安全性	性能	推荐场景
off	 低	 最高	本地测试
non-main	 中	 高	日常使用
docker	 高	 中	生产环境
vm	  最高	 低	高安全要求

9.5 命令执行限制

白名单模式（推荐）

```
{
  "tools": {
    "terminal": {
      "enabled": true,
      "mode": "whitelist",
      "allowedCommands": [
        "ls", "cd", "pwd", "cat", "head", "tail",
        "grep", "find", "wc", "sort", "uniq",
        "git", "npm", "node", "python3"
      ],
      "allowedPaths": [
        "/home/user/projects",
        "/tmp/openclaw"
      ]
    }
  }
}
```

黑名单模式

```
{
  "tools": {
    "terminal": {
      "enabled": true,
      "mode": "blacklist",
      "blockedCommands": [
        "rm -rf /",
        "dd",
        "mkfs",

```

```
        "fdisk",
        "sudo",
        "su"
    ]
}
}
```

9.6 API 限额与预算控制

设置硬性限额

```
{
  "models": {
    "budget": {
      "dailyLimit": 10.00,
      "monthlyLimit": 100.00,
      "alertThreshold": 0.8,
      "action": "pause"
    }
  }
}
```

在提供商侧设置限额

Anthropic:

1. 访问 <https://console.anthropic.com>
2. Settings → Usage Limits
3. 设置每月最大消费

OpenAI:

1. 访问 <https://platform.openai.com>
2. Settings → Usage limits
3. 设置硬性限额

9.7 日志与审计

启用详细日志

```
{
  "logging": {
    "level": "verbose",
```

```
"file": "/var/log/openclaw/gateway.log",
"maxSize": "100M",
"maxFiles": 10,
"audit": {
  "enabled": true,
  "file": "/var/log/openclaw/audit.log",
  "events": [
    "auth.success",
    "auth.failure",
    "command.execute",
    "skill.invoke",
    "api.request"
  ]
}
}
```

定期审计

```
# 查看认证日志
grep "auth" /var/log/openclaw/audit.log

# 查看命令执行日志
grep "command.execute" /var/log/openclaw/audit.log

# 统计 API 使用
openclaw stats --api-usage
```

9.8 安全检查清单

部署前检查：

```
# 运行安全审计
openclaw security audit --deep

# 检查配置
openclaw config validate

# 检查权限
openclaw permissions check

# 检查依赖
openclaw deps audit
```

安全评分：

-  90-100：优秀
-  70-89：良好
-  50-69：一般
-  <50：危险

9.9 数据加密

加密存储敏感信息

```
# 使用加密工具
echo "my-api-key" | openssl enc -aes-256-cbc -a -salt -pass pass:my-secret-pass

# 在配置中使用
{
  "apiKey": "U2FsdGVkX1+xxx..." // 加密后的密钥
}
```

加密传输

```
{
  "gateway": {
    "tls": {
      "enabled": true,
      "cert": "/path/to/cert.pem",
      "key": "/path/to/key.pem",
      "ca": "/path/to/ca.pem"
    }
  }
}
```

9.10 应急响应

发现泄露时的应对措施

```
# 1. 立即撤销 API Key
openclaw revoke-key --provider anthropic

# 2. 轮换所有密钥
openclaw rotate-keys --all
```



```
# 3. 审查访问日志
openclaw audit --since "1 hour ago"

# 4. 检查异常活动
openclaw security check --anomaly

# 5. 暂停服务 (如需要)
systemctl stop openclaw-gateway
```

10. 实战应用场景

10.1 场景一：智能邮件助手

需求：自动分类邮件、提取关键信息、生成回复建议

配置步骤

1. 安装邮件技能

```
openclaw skills install email-assistant
```

2. 配置 Gmail API

```
openclaw configure --section skills.email-assistant
```

```
{
  "skills": {
    "email-assistant": {
      "provider": "gmail",
      "credentials": {
        "clientId": "your-client-id",
        "clientSecret": "your-client-secret",
        "refreshToken": "your-refresh-token"
      },
    },
    "rules": {
      "categories": {
        "work": {
          "from": ["@company.com"],
          "labels": ["工作", "重要"]
        },
      },
      "newsletter": {
        "subject": ["newsletter", "订阅"],
      },
    },
  },
}
```

```
    "action": "archive"
  }
}
}
}
}
```

3. 使用示例

用户：检查邮件并总结重要的

AI：[正在检查邮件...]

 收到 15 封新邮件

 工作邮件：3 封

– 来自：boss@company.com

主题：项目进度更新

摘要：要求本周五前提交报告

 订阅邮件：10 封（已归档）

 建议回复：

收到，我会按时提交报告。

10.2 场景二：代码审查助手

需求：自动审查 Pull Request，提供改进建议

配置步骤

1. 安装 GitHub 技能

```
openclaw skills install github-reviewer
```

2. 配置 GitHub Token

```
openclaw configure --section skills.github-reviewer
```

```
{
  "skills": {
    "github-reviewer": {
      "token": "your-github-token",
      "repositories": [
        "owner/repo1",
```

```

    "owner/repo2"
  ],
  "reviewRules": {
    "maxFileChanges": 500,
    "requireTests": true,
    "checkStyle": true
  }
}
}
}

```

3. 设置自动审查

```

{
  "automations": {
    "github-pr-review": {
      "trigger": "github.pull_request.opened",
      "action": "skill:github-reviewer.review",
      "filters": {
        "repository": ["owner/repo"],
        "author": ["exclude-bot"]
      }
    }
  }
}
}

```

4. 使用示例

用户：审查 #123 PR

AI：[正在审查 PR #123...]

 代码变更统计：

- 文件数：5
- 新增行：+234
- 删除行：-56
- 改动行：+12/-8

 优点：

- 代码结构清晰
- 添加了单元测试
- 文档完善

 建议改进：

1. src/utils/helper.js:45
建议使用 `const` 而非 `let`

2. tests/test.js:78

测试覆盖率不足，建议增加边界情况测试

🔍 潜在问题：

- 发现一处内存泄漏风险 (line 120)

💬 审查意见：总体良好，建议修复上述问题后再合并。

10.3 场景三：社交媒体管理

需求：自动发布内容、分析数据、定时任务

配置步骤

1. 安装社交媒体技能

```
openclaw skills install social-media-manager
```

2. 配置平台凭证

```
openclaw configure --section skills.social-media
```

```
{
  "skills": {
    "social-media-manager": {
      "platforms": {
        "twitter": {
          "enabled": true,
          "apiKey": "your-api-key",
          "apiSecret": "your-api-secret",
          "accessToken": "your-access-token",
          "accessSecret": "your-access-secret"
        },
        "telegram": {
          "enabled": true,
          "botToken": "your-bot-token",
          "channelId": "@your-channel"
        }
      },
      "schedules": {
        "morning-post": {
          "time": "09:00",
          "timezone": "Asia/Shanghai",
```

```
        "content": "早上好! 今天你想学什么?"
    }
}
}
}
}
```

3. 使用示例

用户: 发布推文: "OpenClaw 真的很棒! #AI #OpenSource"

AI: [正在发布推文...]

✅ 已成功发布!

📈 预计效果:

- 展示量: 500-1000
- 互动率: 2-5%

💡 建议:

- 添加图片可以提高 30% 互动率
- 最佳发布时间: 12:00-14:00

10.4 场景四: 个人知识管理

需求: 自动整理笔记、建立知识图谱、智能检索

配置步骤

1. 安装知识管理技能

```
openclaw skills install knowledge-base
```

2. 配置存储路径

```
{
  "skills": {
    "knowledge-base": {
      "storagePath": "~/KnowledgeBase",
      "indexing": {
        "enabled": true,
        "interval": "hourly"
      },
    },
    "tags": {
      "autoTag": true,
      "similarityThreshold": 0.8
    }
  }
}
```

```
}  
}  
}  
}
```

3. 使用示例

用户：记录：今天学习了 OpenClaw 的记忆系统

AI:  已记录到知识库

 自动标签：

- #OpenClaw
- #记忆系统
- #AI

 相关笔记：

- "Agent 架构设计"
- "记忆存储方案对比"

 建议：

可以继续了解：

1. 记忆压缩算法
2. 语义搜索实现

10.5 场景五：自动化 workflow

需求：定时任务、事件触发、多步骤自动化

配置示例

1. 每日报表生成

```
{  
  "automations": {  
    "daily-report": {  
      "schedule": "0 18 * * 1-5",  
      "actions": [  
        {  
          "type": "skill",  
          "name": "fetch-data",  
          "params": {  
            "source": "analytics",  
            "metrics": ["visitors", "conversions"]  
          }  
        },  
      ],  
    },  
  },  
}
```

```

    {
      "type": "skill",
      "name": "generate-report",
      "params": {
        "template": "daily"
      }
    },
    {
      "type": "channel",
      "name": "telegram",
      "params": {
        "chat": "my-channel",
        "message": "${report}"
      }
    }
  ]
}

```

2. 事件驱动的自动化

```

{
  "automations": {
    "on-new-email": {
      "trigger": "email.received",
      "condition": {
        "from": "boss@company.com",
        "subject": "紧急"
      },
      "actions": [
        {
          "type": "notification",
          "message": "收到老板的紧急邮件！"
        },
        {
          "type": "memory",
          "action": "remember",
          "content": "紧急事项: ${email.subject}"
        }
      ]
    }
  }
}

```

10.6 场景六：数据监控与告警

需求： 监控网站、API、服务器状态

配置步骤

1. 安装监控技能

```
openclaw skills install site-monitor
```

2. 配置监控目标

```
{
  "skills": {
    "site-monitor": {
      "targets": [
        {
          "name": "公司官网",
          "url": "https://example.com",
          "interval": 300,
          "timeout": 10,
          "expectedStatus": 200
        },
        {
          "name": "API 端点",
          "url": "https://api.example.com/health",
          "interval": 60,
          "timeout": 5,
          "expectedResponse": {"status": "ok"}
        }
      ],
      "alerts": {
        "channels": ["telegram", "email"],
        "escalation": {
          "5min": "warning",
          "15min": "critical"
        }
      }
    }
  }
}
```

3. 使用示例

用户：检查所有监控目标状态

AI：[正在检查...]

- ✅ 公司官网：正常（响应时间：120ms）
- ✅ API 端点：正常（响应时间：45ms）
- ⚠️ 数据库服务：响应慢（响应时间：2.3s）

💬 已发送告警到 Telegram

10.7 十个真实使用场景

来自社区的真实案例：

1. **飞书群管理** - 自动回答常见问题、整理群文件
2. **代码调试** - 分析错误日志、提供解决方案
3. **文献阅读** - 提取论文要点、生成总结
4. **投资分析** - 监控股票、生成财报分析
5. **学习助手** - 制定学习计划、跟踪进度
6. **健康管理** - 记录症状、分析趋势
7. **旅行规划** - 推荐行程、预订酒店
8. **项目管理** - 跟踪任务、生成报告
9. **客户服务** - 自动回复、工单分类
10. **个人 CRM** - 管理联系人、记录互动

11. 故障排除与FAQ

11.1 常见问题

Q1: Gateway 启动失败

症状：

```
$ openclaw gateway
Error: Port 18789 already in use
```

解决方案：

```
# 1. 查看端口占用
sudo lsof -i :18789
```

```
# 2. 杀死占用的进程
kill -9 <PID>

# 3. 或更改端口
openclaw gateway --port 18790
```

Q2: AI 不回复消息

可能原因：

1. 未配置 API 认证
2. API Key 无效
3. 未完成配对

排查步骤：

```
# 1. 检查认证
openclaw health

# 2. 测试 API 连接
openclaw test api

# 3. 查看配对状态
openclaw pairing list

# 4. 查看日志
journalctl -u openclaw-gateway -f
```

Q3: 记忆系统不工作

症状： AI 不记住之前说过的话

解决方案：

```
# 1. 检查记忆目录权限
ls -la ~/.openclaw/memory/

# 2. 检查配置
openclaw config get memory.enabled

# 3. 手动测试记忆
echo "测试：记住我喜欢苹果" | openclaw message --local
```

Q4: WhatsApp 频繁断线

原因：WhatsApp 有连接时长限制

解决方案：

```
# 1. 配置自动重连
{
  "channels": {
    "whatsapp": {
      "reconnect": {
        "enabled": true,
        "interval": 300,
        "maxRetries": 10
      }
    }
  }
}

# 2. 或使用守护进程保持连接
systemctl enable openclaw-gateway
```

Q5: 技能加载失败

症状：

```
Error: Cannot find module 'xxx'
```

解决方案：

```
# 1. 重新安装技能依赖
cd ~/.openclaw/skills/xxx
npm install

# 2. 检查 Node 版本
node --version # 需要 >= 22

# 3. 清除缓存
npm cache clean --force
```

11.2 性能优化

问题：响应速度慢

优化方案：

1. 使用更快的模型

```
{
  "agents": {
    "main": {
      "model": {
        "provider": "anthropic",
        "model": "claude-haiku-4-5-20250929" // 比 Sonnet 快
      }
    }
  }
}
```

2. 减少记忆容量

```
{
  "memory": {
    "maxTokens": 4000, // 降低到 4000
    "summaryThreshold": 3000
  }
}
```

3. 启用缓存

```
{
  "cache": {
    "enabled": true,
    "ttl": 3600,
    "maxSize": "100MB"
  }
}
```

问题：内存占用过高

解决方案：

```
# 1. 限制 Node.js 内存
node --max-old-space-size=512 $(which openclaw) gateway

# 2. 定期重启
crontab -e
# 添加: 0 3 * * * systemctl restart openclaw-gateway
```

```
# 3. 优化日志配置
{
  "logging": {
    "level": "warn", // 降低日志级别
    "maxSize": "50M"
  }
}
```

11.3 日志分析

查看日志

```
# 实时日志
journalctl -u openclaw-gateway -f

# 最近100行
journalctl -u openclaw-gateway -n 100

# 按时间过滤
journalctl -u openclaw-gateway --since "1 hour ago"

# 按关键词过滤
journalctl -u openclaw-gateway | grep "ERROR"
```

日志级别

级别	说明	示例
DEBUG	详细调试信息	函数调用、变量值
INFO	一般信息	启动、关闭
WARN	警告信息	配置过时、性能问题
ERROR	错误信息	API 失败、连接中断

11.4 调试模式

启用详细日志

```
# 启动时添加 --verbose
openclaw gateway --verbose

# 或 --debug
openclaw gateway --debug
```

单步调试

```
# 测试单个功能
openclaw test api
openclaw test memory
openclaw test skills

# 模拟消息
openclaw message send --target telegram --message "测试消息"
```

11.5 配置验证

```
# 验证配置文件
openclaw config validate

# 检查配置语法
cat ~/.openclaw/openclaw.json | jq .

# 测试配置
openclaw config test
```

11.6 完整诊断流程

```
# 1. 系统健康检查
openclaw health

# 2. 详细状态
openclaw status --all

# 3. 安全审计
openclaw security audit --deep

# 4. 依赖检查
openclaw deps check

# 5. 配置验证
openclaw config validate

# 6. 生成诊断报告
openclaw diagnostic > report.txt
```

12. 进阶主题

12.1 多 Agent 协作

OpenClaw 支持运行多个 Agent，每个 Agent 有独立的配置和职责。

配置多 Agent

```
{
  "agents": {
    "main": {
      "role": "通用助手",
      "model": "claude-sonnet-4-5",
      "tools": ["web", "browser", "terminal"]
    },
    "coder": {
      "role": "代码专家",
      "model": "claude-opus-4-5",
      "tools": ["code", "terminal", "github"],
      "workspace": "~/.openclaw/coder-workspace"
    },
    "writer": {
      "role": "写作助手",
      "model": "claude-sonnet-4-5",
      "tools": ["web", "memory"],
      "personality": "creative"
    }
  }
}
```

Agent 路由

```
{
  "routing": {
    "strategy": "keyword",
    "rules": {
      "coder": ["代码", "编程", "debug", "bug"],
      "writer": ["写作", "文章", "博客"]
    }
  }
}
```

使用示例

用户: @coder 帮我写一个 Python 脚本

Coder Agent: 你好, 我是代码专家。我来帮你写脚本...
[生成代码]

用户: @writer 写一篇技术博客

Writer Agent: 好的, 我来帮你撰写...
[生成文章]

12.2 自定义模型提供商

OpenClaw 支持添加自定义 LLM 提供商。

添加 OpenAI 兼容 API

```
{
  "models": {
    "providers": {
      "custom-llm": {
        "type": "openai-compatible",
        "baseUrl": "https://api.custom-llm.com/v1",
        "apiKey": "your-api-key",
        "models": {
          "model-name": "custom-model-v1"
        }
      }
    }
  }
}
```

添加 Anthropic 兼容 API

```
{
  "models": {
    "providers": {
      "custom-claude": {
        "type": "anthropic-compatible",
        "baseUrl": "https://api.custom-claude.com",
        "apiKey": "your-api-key",
        "version": "2023-06-01"
      }
    }
  }
}
```


12.3 远程部署

Tailscale 部署

```
# 1. 安装 Tailscale
curl -fsSL https://tailscale.com/install.sh | sh

# 2. 连接网络
sudo tailscale up

# 3. 获取 Tailscale IP
tailscale ip -4

# 4. 配置 OpenClaw 监听 Tailscale IP
{
  "gateway": {
    "host": "100.x.x.x", // Tailscale IP
    "port": 18789
  }
}
```

Cloudflare Workers 部署

```
// worker.js
export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    // 代理到 OpenClaw Gateway
    const gatewayUrl = "http://your-server:18789";

    // 转发请求
    const modifiedRequest = new Request(gatewayUrl + url.pathname, {
      method: request.method,
      headers: request.headers,
      body: request.body
    });

    return fetch(modifiedRequest);
  }
}
```

12.4 监控与可观测性

Prometheus 指标

```
{
  "metrics": {
    "enabled": true,
    "port": 9090,
    "path": "/metrics"
  }
}
```

Grafana 仪表板

```
{
  "dashboard": {
    "title": "OpenClaw Monitoring",
    "panels": [
      {
        "title": "API 调用次数",
        "targets": [{
          "expr": "openclaw_api_calls_total"
        }]
      },
      {
        "title": "平均响应时间",
        "targets": [{
          "expr": "openclaw_response_time_avg"
        }]
      }
    ]
  }
}
```

12.5 插件开发

创建自定义插件扩展功能：

```
// plugins/my-plugin.js
class MyPlugin {
  constructor(config) {
    this.config = config;
  }

  async onLoad(agent) {
    console.log('Plugin loaded!');
  }
}
```

```

    // 注册钩子
    agent.hooks.on('message.pre', this.onMessage.bind(this));
  }

  async onMessage(context) {
    // 处理消息前的钩子
    console.log('Received:', context.message);
    return context;
  }

  async onUnload() {
    console.log('Plugin unloaded!');
  }
}

module.exports = MyPlugin;

```

12.6 国际化

配置中文支持

```

{
  "localization": {
    "locale": "zh-CN",
    "timezone": "Asia/Shanghai",
    "dateFormat": "YYYY-MM-DD",
    "currency": "CNY"
  }
}

```

自定义翻译

```

// i18n/zh-CN.json
{
  "greeting": "你好! ",
  "farewell": "再见! ",
  "error": "发生错误: {error}"
}

```

13. 社区资源汇总

13.1 官方资源

资源	链接	说明
官方文档	https://docs.openclaw.ai	最权威的文档
GitHub 仓库	https://github.com/openclaw/openclaw	源代码
官方博客	https://blog.openclaw.ai	更新公告
官方 Discord	https://discord.gg/openclaw	社区讨论
官方 X/Twitter	@openclaw	实时动态

13.3 社区项目

核心项目

- **OpenClaw** - 主项目
- **ClawHub** - 技能市场
- **OpenClaw-Feishu** - 飞书插件
- **OpenClaw-Installer** - 一键安装脚本

社区插件

- **openclaw-wechat** - 微信集成
- **openclaw-dingtalk** - 钉钉集成
- **openclaw-slack** - Slack 增强
- **openclaw-teams** - Microsoft Teams

实用工具

- **oneclaw** - 多实例管理
- **clawshell** - Web 终端
- **clawmon** - 监控面板
- **clawbench** - 性能测试

13.4 学习资源

视频教程

- YouTube: 搜索 "OpenClaw Tutorial"
- Bilibili: <https://www.bilibili.com/search?keyword=OpenClaw>
- 官方视频系列: <https://www.youtube.com/@openclaw>

13.5 获取帮助

官方支持

- **GitHub Issues:** 报告 Bug
- **Discord:** 实时讨论
- **Email:** support@openclaw.ai

社区支持

- **Reddit:** r/OpenClaw
- **Stack Overflow:** 标签 openclaw
- 中文社区: 微信群、QQ群

商业支持

- 官方合作伙伴: 提供企业支持
- 咨询公司: OpenClaw 部署服务
- 培训课程: 线上/线下培训

13.6 贡献指南

如何贡献

1. **Fork 仓库**
2. 创建分支 (`git checkout -b feature/AmazingFeature`)
3. 提交更改 (`git commit -m 'Add some AmazingFeature'`)
4. 推送分支 (`git push origin feature/AmazingFeature`)
5. 创建 **Pull Request**

贡献类型

- 🐛 Bug 修复
- ✨ 新功能
- 📝 文档改进
- 🎨 代码重构
- ⚡ 性能优化
- ✅ 测试覆盖

行为准则

- 尊重他人

- 建设性反馈
 - 遵守开源协议
 - 保护隐私
-

附录

A. 配置文件完整示例

```
{
  "version": "2026.2.8",
  "gateway": {
    "port": 18789,
    "host": "127.0.0.1",
    "auth": {
      "enabled": true,
      "token": "your-secure-token-here",
      "ipWhitelist": ["127.0.0.1", "192.168.1.0/24"]
    }
  },
  "agents": {
    "main": {
      "workspace": "~/.openclaw/workspace",
      "model": {
        "provider": "anthropic",
        "model": "claude-sonnet-4-5-20250929",
        "maxTokens": 100000,
        "temperature": 0.7
      },
      "memory": {
        "enabled": true,
        "maxTokens": 8000,
        "storagePath": "~/.openclaw/memory",
        "summaryThreshold": 5000
      },
      "tools": ["web", "browser", "terminal", "memory"],
      "sandbox": {
        "mode": "non-main"
      }
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
```

```

    "botToken": "your-telegram-bot-token"
  },
  "discord": {
    "enabled": false
  },
  "whatsapp": {
    "enabled": false
  }
},
"skills": {
  "web-search": {
    "enabled": true,
    "provider": "brave",
    "apiKey": "your-brave-api-key"
  }
},
"logging": {
  "level": "info",
  "file": "/var/log/openclaw/gateway.log",
  "audit": {
    "enabled": true,
    "file": "/var/log/openclaw/audit.log"
  }
},
"security": {
  "sandboxMode": "docker",
  "commandWhitelist": ["ls", "cat", "grep"],
  "apiBudget": {
    "daily": 10.00,
    "monthly": 100.00
  }
}
}

```

B. 常用命令速查

```

# 安装与更新
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw update

# 配置
openclaw onboard
openclaw configure --section <section>
openclaw config validate

```

```
# 服务管理
openclaw gateway [--port 18789] [--verbose]
systemctl start/stop/restart openclaw-gateway

# 状态检查
openclaw status
openclaw health
openclaw security audit --deep

# 消息测试
openclaw message send --target <channel> --message "<text>"

# 技能管理
openclaw skills list
openclaw skills install <skill>
openclaw skills update <skill>
openclaw skills uninstall <skill>

# 日志查看
journalctl -u openclaw-gateway -f
openclaw logs --tail 100

# 配对管理
openclaw pairing list <channel>
openclaw pairing approve <channel> <code>
```

C. 端口与防火墙

端口	用途	外部访问
18789	Gateway API	可选（需认证）
3000	Dashboard	可选（需反向代理）
9090	Prometheus 指标	内网

防火墙配置：

```
# UFW (Ubuntu)
sudo ufw allow 18789/tcp
sudo ufw enable

# firewalld (CentOS)
sudo firewall-cmd --add-port=18789/tcp --permanent
sudo firewall-cmd --reload
```


D. 目录结构

```
~/openclaw/
├─ openclaw.json          # 主配置文件
├─ credentials/          # 凭证目录
│   └─ oauth.json        # OAuth 凭证
├─ agents/               # Agent 配置
│   └─ main/
│       ├── agent.json
│       └─ auth-profiles.json
├─ workspace/            # 工作区
│   ├── SOUL.md           # AI 人格
│   ├── USER.md           # 用户画像
│   └─ skills/            # 用户技能
├─ memory/               # 记忆存储
│   ├── 2026-01-15.md
│   └─ MEMORY.md
├─ skills/               # 全局技能
│   ├── web-search/
│   ├── browser/
│   └─ ...
├─ logs/                 # 日志文件
└─ cache/                # 缓存目录
```

E. 环境变量

```
# API Keys
export ANTHROPIC_API_KEY="sk-ant-xxx"
export OPENAI_API_KEY="sk-xxx"
export BRAVE_SEARCH_API_KEY="xxx"

# Gateway 配置
export OPENCLAW_PORT=18789
export OPENCLAW_HOST="127.0.0.1"
export OPENCLAW_TOKEN="your-token"

# Node.js 配置
export NODE_ENV="production"
export NODE_OPTIONS="--max-old-space-size=1024"

# 日志配置
export OPENCLAW_LOG_LEVEL="info"
export OPENCLAW_LOG_FILE="/var/log/openclaw/gateway.log"
```

结语

OpenClaw 代表了 AI Agent 的未来方向——本地优先、隐私可控、完全开源。通过本手册，你应该能够：

- ✅ 理解 OpenClaw 的核心理念和架构
- ✅ 完成从安装到配置的全过程
- ✅ 连接多种聊天平台
- ✅ 掌握记忆系统和技能系统
- ✅ 配置安全的生产环境
- ✅ 应用到实际工作场景

记住：OpenClaw 是一个快速发展的项目，本手册内容可能随版本更新而变化。请以官方文档为准。

祝你使用 OpenClaw 愉快！🦀

附录：OpenClaw 大事记

2025-11-15 Clawbot/Clawdbot 项目启动
2025-12-20 GitHub 公开发布，24小时9,000 Stars
2026-01-26 收到 Anthropic 法务建议函
2026-01-27 改名 Moltbot
2026-01-29 最终定名 OpenClaw
2026-02-01 GitHub Stars 突破 100,000
2026-02-10 发布 v2026.2.8，技能生态爆发
2026-02-15 本手册发布

感谢以下人员和组织对 OpenClaw 生态的贡献：

- Peter Steinberger（创始人）
- OpenClaw 核心开发团队
- 全球 500+ 贡献者
- 1700+ 技能开发者
- 所有测试者和用户

让我们一起构建OpenClaw的未来！🚀